

RoboRT: A Real-Time C++ Inference Engine for pi0.5 Vision-Language-Action Policies

liuwang
Baidu

Abstract

Vision-language-action (VLA) models such as pi0.5 have demonstrated strong performance in robot manipulation, yet their inference latency remains the dominant bottleneck to real-time closed-loop control. We present RoboRT, a native C++ inference engine tailored to the flow-matching denoising schedule of pi0.5. Rather than adapting a generic large-language-model (LLM) runtime, RoboRT implements a hand-written GGUF reader that loads F32, BF16, and quantized weights directly, a compute graph that is cached and rebuilt only when the input shape changes, and an engine-owned noise RNG that renders every prediction bit-exact reproducible. On an A10 GPU, RoboRT achieves **161 ms/step** (6.2 Hz), a **2.3×** speedup over the LeRobot PyTorch reference at 377 ms/step (2.7 Hz) under identical observation and seed, while retaining 38/40 task success on the LIBERO benchmark. RoboRT is released under the Apache-2.0 license together with a deterministic self-check tool that enables bit-exact comparison across implementations.

1 Introduction

Recent vision-language-action (VLA) policies, exemplified by the pi0 family [1], jointly model vision, language, and action in a single architecture and have become a de facto baseline for generalist robot manipulation. A VLA policy must keep pace with the control loop it drives: higher inference throughput directly translates into higher task success in dynamic manipulation. The practical target is on the order of 10 Hz per inference step; anything slower necessitates aggressive motion smoothing and degrades closed-loop behavior.

Despite their appeal, deploying these models at interactive rates is challenging for three reasons. First, *reference implementations are slow*: the widely used LeRobot [3] stack runs pi0.5 at 377 ms/step on an A10, sustaining only 2.7 Hz—a small fraction of the 10 Hz target—and leaving little headroom for a reactive controller. Second, *generic LLM runtimes are ill-suited to the workload*: mature inference engines are optimized for autoregressive token generation with continuous batching, speculative decoding, and KV-cache paging, whereas pi0.5 predicts action chunks through an iterative flow-matching denoising loop [4] in which the entire chunk is denoised in parallel over a fixed number of steps, all sharing a single precomputed visual-language prefix. Grafting the model onto such a runtime introduces substantial unused machinery and a scheduling model that is fundamentally mismatched to the denoising workload. Third, *the deployment chain is fragmented*: vision encoding, prefix caching, noise scheduling, and action denormalization are usually assembled from separate, independently maintained implementations, rendering results difficult to reproduce and still more difficult to compare across codebases.

We address these limitations with **RoboRT**, a self-contained C++ inference engine for pi0.5. The engine is native rather than an adapter over an LLM framework: it reads weights with a hand-written GGUF reader, executes a compute graph that is cached across steps and rebuilt only when

the input shapes change, and maintains the noise RNG as engine state, rendering every prediction bit-exact reproducible from a fixed seed. RoboRT builds solely on generic compute primitives—the ggml graph runtime with its CPU/CUDA backends and the SigLIP [8] vision tower—and contains no LLM-specific layers.

At 161 ms/step (6.2 Hz), RoboRT closes more than half of the gap toward interactive control rates on the same A10 hardware, and its per-phase breakdown shows that denoising—not vision encoding—dominates, indicating clear headroom for further optimization (Sec. 5.2).

Our contributions are:

- **A native pi0.5 inference engine** (Sec. 3): a hand-written GGUF reader, a cached compute graph, and a flow-matching denoising loop, with no dependence on an LLM runtime.
- **A reproducibility mechanism** (Sec. 4): deterministic engine-state RNG semantics and a self-check tool that emits 1,600 floats bit-exactly for cross-implementation regression.
- **Measured results** (Sec. 5): 161 ms/step on an A10 (2.3× over LeRobot) and 38/40 task success on LIBERO with seed 42, consistent with the reference policy.

2 Related Work

pi0 and pi0.5. pi0 [1] is an open-source VLA built on the PaliGemma vision-language backbone with a flow-matching action expert; pi0.5 [2] extends it with a stronger backbone while retaining the same denoising interface. Both predict a chunk of actions (the suffix) by iteratively denoising Gaussian noise conditioned on vision, language, and proprioceptive state.

VLA inference stacks. LeRobot [3] provides the canonical PyTorch implementation of pi0 and pi0.5 and serves as the reference against which we compare. Its latency is dominated by Python overhead and eager execution. Serving-oriented runtimes (e.g., SGLang [6]) offer request batching and continuous KV-cache management, but they target autoregressive LLM serving rather than the non-autoregressive denoising loop of action experts.

Flow matching and diffusion inference. Flow matching [4] and diffusion models [5] share an iterative refinement schedule. Production inference for these models optimizes the number of steps, the schedule, and the per-step compute—the same design knobs that RoboRT exposes—rather than token-level batching.

Graph runtimes and vision encoders. ggml [7] provides a dependency-free computation-graph runtime with CPU and CUDA backends. We use only its generic operators and implement all pi0.5-specific graph construction ourselves. The vision tower is the SigLIP [8] encoder, executed as a standalone graph whose output is incorporated into the language-model prefix.

Benchmarks. LIBERO [9] is a suite of scripted long-horizon manipulation tasks that has become a standard benchmark for evaluating VLA policies; we use it to confirm that RoboRT preserves the behavior of the reference policy end to end.

3 Method

Figure 1 illustrates the end-to-end data flow. The engine is organized as a single library (`vla-pi05`) with a public C++ API: a `Config` describing the model dimensions, an `Inputs` struct carrying the observation, and a `predict` function returning a flat $n_{\text{steps}} \times d_{\text{act}}$ action chunk.

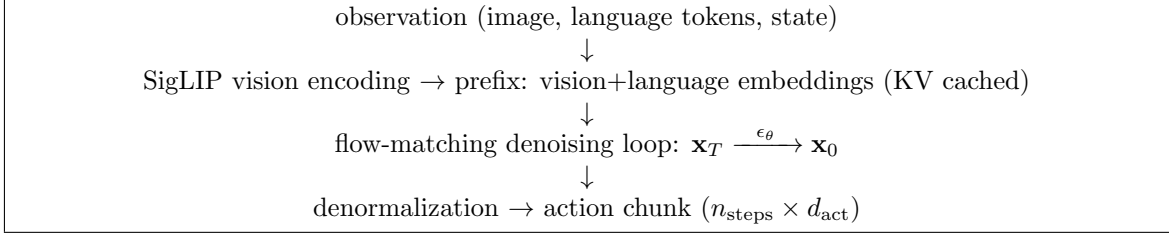


Figure 1: End-to-end data flow of RoboRT for one pi0.5 predict.

3.1 Hand-written GGUF reader

pi0.5 checkpoints are distributed as HuggingFace safetensors. RoboRT converts them offline to GGUF with two Python scripts (`convert_pi05_to_gguf.py` and `convert_pi05_mmproj_to_gguf.py`), then loads them at startup with a hand-written reader over the ggml `gguf_*` API. The reader supports F32, BF16, and quantized dtypes and maps tensors directly onto the compute graph without a model-level abstraction layer. The GGUF metadata is the single source of configuration: the model dimensions, the action normalization mode (quantile or mean/std), and the denoising step count are read from the weight file rather than hard-coded into the runtime, so that retraining or re-quantization never requires recompilation.

3.2 Cached compute graph

The inference graph has three shape-dependent inputs: the number of visual tokens, the number of language tokens, and the suffix length. RoboRT caches the graph keyed on this triple: the graph is built once per unique shape and rebuilt only when one of the three changes (e.g., a different language instruction). All n_{steps} iterations of one `predict` share a single graph, and later `predict` calls with unchanged shapes reuse it directly, so the steady-state inference path incurs zero graph-construction overhead.

3.3 Flow-matching denoising loop

Given a noise tensor $\mathbf{x}_T \sim \mathcal{N}(0, I)$, the action expert is iterated as

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Delta t \cdot \epsilon_\theta(\mathbf{x}_t, c), \quad (1)$$

where ϵ_θ is the denoising network conditioned on c (vision, language, and state) and $\Delta t = 1/n_{\text{steps}}$. The prefix (vision + language embeddings) is computed once, and its key/value cache is reused across all denoising iterations. After the loop, the action chunk is denormalized within the engine using the training statistics (quantile or mean/std, per the GGUF metadata) and truncated to the real action dimension d_{act} .

4 Reproducibility

Stochasticity in flow-matching inference originates from the noise sample; hence, reproducibility reduces to controlling the noise RNG. In RoboRT, the RNG is maintained as engine state: it advances once per `predict`, and the environment variable `VLA_PI05_SEED` sets only the starting point of the sequence. The engine provides `vla-pi05-selfcheck`, a deterministic tool that runs with a fixed seed and emits the final prediction’s 1,600 floats. This enables bit-exact comparison

Table 1: End-to-end latency per inference step on an A10 (fp16).

Engine	Vision	Denoise	Other	Total	Speedup
LeRobot (PyTorch)	—	—	—	377 ms	1.0×
RoboRT (C++)	23.7 ms	126.9 ms	1.6 ms	161 ms	2.3×

Table 2: LIBERO task success (seed offset 42, one episode per task).

Engine	Success	Failed tasks
LeRobot (reference)	38/40	{24, 29}
RoboRT (C++)	38/40	{24, 29}

across implementations: any regression in graph construction, operator selection, or normalization manifests as a numerical discrepancy in the emitted values. For a faithful comparison, the server must be queried from a clean process; any additional request advances the noise sequence and alters the marginal task outcomes.

The serving layer additionally enforces the input contract strictly: the state length must equal the configured dimension, language token counts must lie in $[1, n_{\text{lang}}]$, the noise shape must match $\{0, n_{\text{suffix}} \times d_{\text{act}}\}$, and any float input containing NaN or Inf is rejected.

5 Experiments

5.1 Setup

We evaluate RoboRT on an A10 GPU (24 GB) with fp16 weights. The model is the pi0.5 checkpoint fine-tuned on LIBERO-90. Latency is measured as the end-to-end `predict` time with a fixed seed (VLA_PI05_SEED=42). The LeRobot reference processes the same observation under the same seed in PyTorch.

5.2 Inference latency

Table 1 reports the end-to-end latency and its phase breakdown. RoboRT achieves 161 ms/step, a 2.3× speedup over the LeRobot reference at 377 ms/step—equivalently 6.2 Hz versus 2.7 Hz, more than halving the gap to the 10 Hz control target. Within RoboRT, denoising dominates, accounting for 126.9 ms (79% of the total); vision encoding accounts for 23.7 ms (15%), and the remaining serving overhead amounts to 1.6 ms. This distribution indicates that further latency reductions must come primarily from the denoising phase: fewer flow-matching steps, operator fusion within the denoising layers, and CUDA-graph capture of the repeated iterations all represent promising directions for future work.

5.3 LIBERO evaluation

We evaluate the policy on the LIBERO [9] suite with one episode per task, a seed offset of 42, and no video recording. RoboRT achieves **38/40** task success, and the two failures (tasks {24, 29}) coincide exactly with those of the LeRobot reference, confirming that the engine reproduces the behavior of the reference policy end to end.

6 Conclusion

We have presented RoboRT, a native C++ inference engine for the pi0.5 vision-language-action policy. By matching the runtime to the flow-matching workload—hand-written GGUF loading, cached compute graphs, and deterministic noise—RoboRT delivers a $2.3\times$ latency reduction over the reference implementation while remaining bit-exact reproducible and behaviorally identical on LIBERO. The engine is released under the Apache-2.0 license with conversion scripts, a ZeroMQ/Protobuf serving layer, and a deterministic self-check tool.

References

- [1] M. Black et al. π_0 : A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.
- [2] S. Nasiriany et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv:2504.09754*, 2025.
- [3] R. Cadene et al. LeRobot: State-of-the-art machine learning for real-world robotics. <https://github.com/huggingface/lerobot>, 2024.
- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *ICLR*, 2023.
- [5] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.
- [6] L. Zheng et al. SGLang: Efficient execution of structured language model programs. *arXiv:2312.07104*, 2023.
- [7] G. Gerganov et al. ggml: Tensor library for machine learning. <https://github.com/ggml-org/ggml>, 2023.
- [8] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. *ICCV*, 2023.
- [9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *NeurIPS*, 2023.